

Informe de Desempeno MPI Document Search Engine

Proyecto Final

1 Resumen

Se presenta un motor de busqueda de documentos en paralelo usando MPI que emplea ponderacion TF-IDF y similitud coseno. El informe se centra en el analisis de rendimiento obtenido en el cluster, apoyado por graficas generadas a partir de los resultados experimentales.

2 Descripcion del algoritmo

El motor transforma cada documento en un vector TF-IDF y calcula un vector TF-IDF para la consulta. Para cada documento se calcula la similitud coseno con la consulta y se ordenan los resultados para obtener el Top-K.

2.1 TF-IDF

Para cada termino t en el documento d :

$$TF(t, d) = \frac{\text{freq}(t, d)}{\text{total de terminos en } d}$$

$$IDF(t) = \log \left(\frac{N}{1 + df(t)} \right)$$

El vector final es $TF-IDF(t, d) = TF(t, d) \cdot IDF(t)$, donde N es el numero total de documentos y $df(t)$ es el numero de documentos que contienen el termino.

2.2 Similitud coseno

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

Los documentos se ordenan de mayor a menor segun esta similitud.

3 Paralelizacion con MPI

El programa divide el trabajo en cuatro fases:

1. **Indexacion (rank 0):** carga el corpus, construye vocabulario, calcula IDF y la matriz TF-IDF.
2. **Distribucion:** el vector de consulta se difunde con `MPI_Bcast` y la matriz TF-IDF se reparte con `MPI_Scatterv`.

3. **Busqueda local:** cada proceso calcula similitudes coseno y obtiene su Top-K local.
4. **Recoleccion y merge:** el master recoge resultados con `MPI_Gatherv` y ordena el Top-K global.

4 Configuración experimental

Las pruebas se realizaron sobre un cluster con ejecución MPI, usando corpus sintéticos generados con `generate_corpus.py`. Se evaluaron corpus de 100, 500, 1000 y 5000 documentos, con vocabulario de alrededor de 5000 palabras. Se usó $\text{Top-K} = 10$ y se ejecutaron múltiples repeticiones por configuración (ver resultados en `results/`).

5 Resultados y análisis

5.1 Tiempo total

La Figura 1 muestra el tiempo total promedio para cada tamaño de corpus. Para hasta 8 procesos el tiempo total es muy similar a la ejecución secuencial; a partir de 16 procesos el tiempo aumenta.

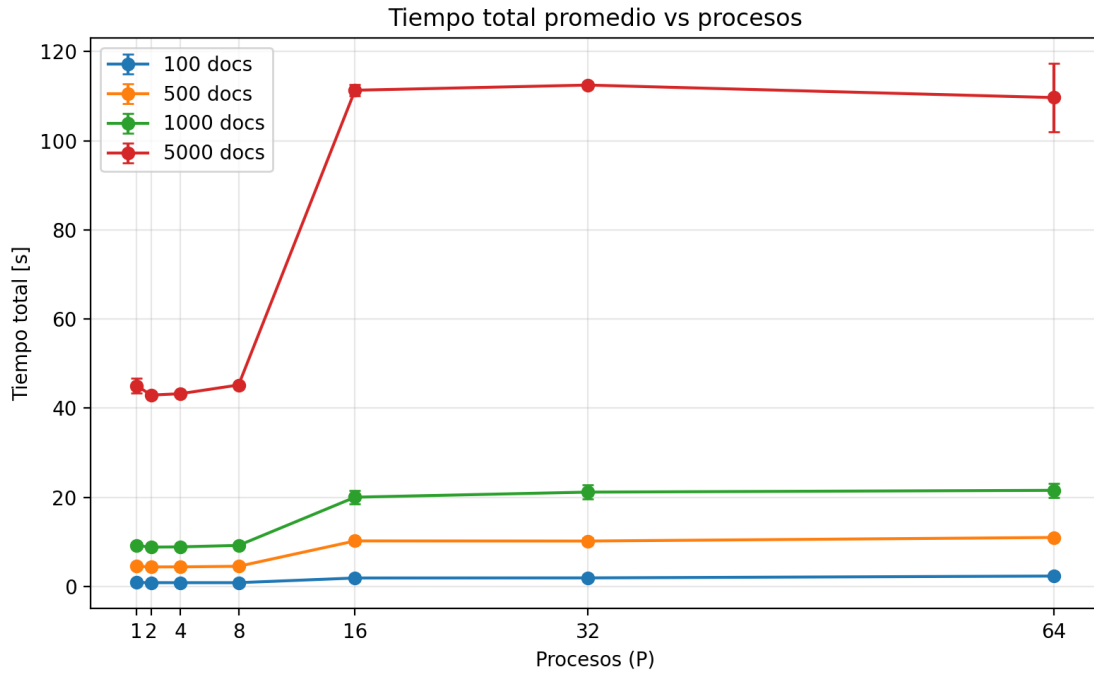


Figure 1: Tiempo total promedio vs procesos.

5.2 Speedup y eficiencia

La Figura 2 muestra que el speedup se mantiene cerca de 1 para $P=2-8$, y cae por debajo de 1 para $P \geq 16$. La eficiencia (Figura 3) decrece rápidamente, lo cual indica fuerte sobrecarga de comunicación y una fracción serial dominante (Amdahl).

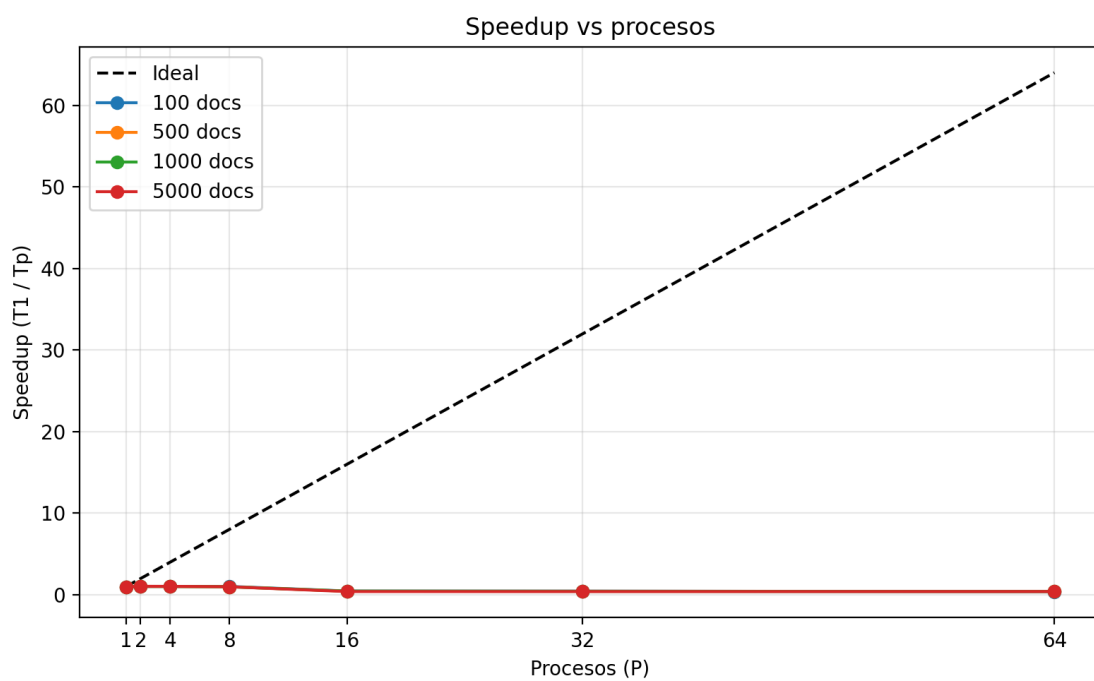


Figure 2: Speedup por cantidad de procesos.

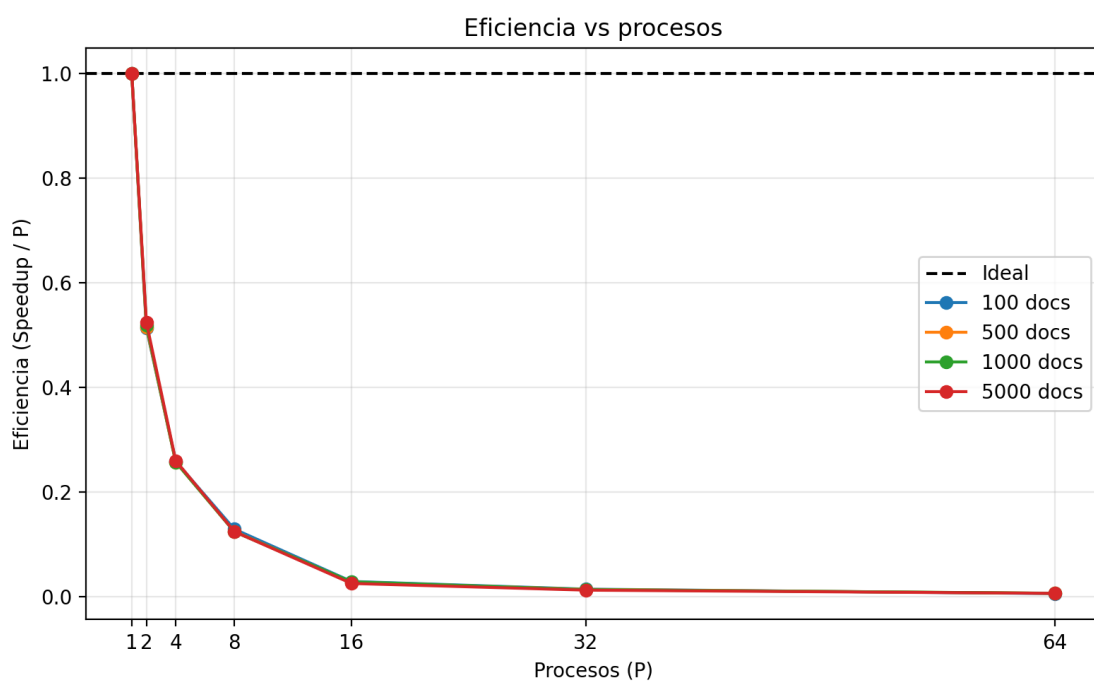


Figure 3: Eficiencia por cantidad de procesos.

5.3 Desglose por fases

La Figura 4 evidencia que la **indexacion** domina el tiempo total en todos los tamanos de corpus. Por ejemplo, para 5000 documentos el tiempo de indexacion representa casi todo el total. Aunque el tiempo de busqueda local disminuye al aumentar P, el beneficio queda oculto por la seccion serial y por el costo de **Scatter/Gather**, que crece con el numero de procesos.

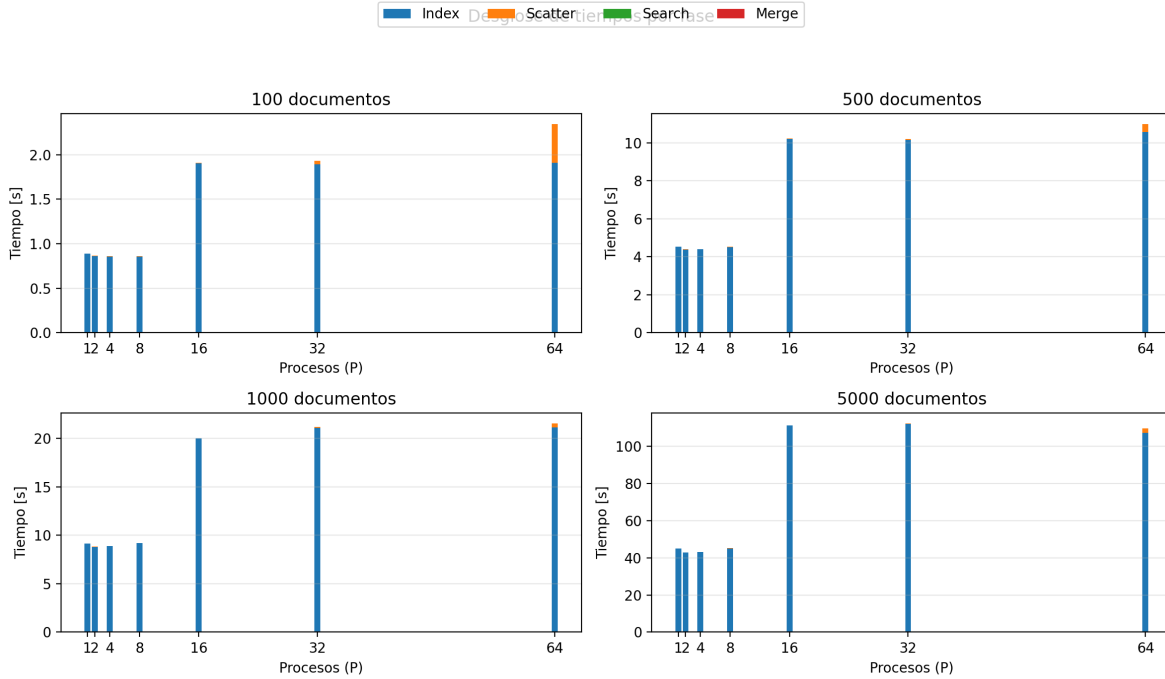


Figure 4: Desglose de tiempos por fase (Index/Scatter/Search/Merge).

5.4 Observaciones clave

- **Dominio serial:** la indexacion ocurre solo en el master y concentra la mayor parte del tiempo.
- **Escalamiento limitado:** el speedup apenas supera 1 para $P=2-4$ y se degrada para $P \geq 16$.
- **Sobrecarga de comunicacion:** el tiempo de **Scatter** crece notablemente en P grandes, afectando el total.

6 Conclusiones

El motor de busqueda paraleliza correctamente la fase de similitud, pero el rendimiento global esta limitado por la indexacion secuencial y la comunicacion MPI. Para mejorar la escalabilidad, el siguiente paso natural seria paralelizar el calculo de TF-IDF e IDF (vocabulario distribuido o map-reduce de terminos) y reducir el volumen de datos transferidos.

7 Como reproducir las graficas

Ejecutar los scripts desde la raiz del proyecto:

```
python3 plot_scaling.py  
python3 plot_breakdown.py
```

Las figuras se guardan en el directorio **figuras/**.